

PHP MVC Project – Master Tasks Backlog

Use `- [x]` to mark completed tasks.

Keep this file in your project root for AI reference and progress tracking.

Phase 0 – Environment & Pre-requirements (before coding)

- ✓ ~~Git~~ installed & configured (`git --version` , `user.name` , `user.email`)
 - ✓ ~~GitHub~~ repo created, cloned locally, `.gitignore` added (`vendor/` , `.env` , `config/db_config.php` , `public/uploads/` , `*.log`)
 - ✓ ~~XAMPP~~ installed, Apache + MySQL running
 - ✓ ~~PHP ≥7.4~~ verify `php -v` , extensions enabled: `pdo_mysql` , `mysql` , `fileinfo` , `session` , `json`
 - ✓ ~~MySQL~~ test connection (root, empty password), create database (e.g., `test_mvc`)
 - ✓ ~~Composer~~ installed (`composer --version`)
 - ✓ ~~Folder structure~~ created (as per SDLC):
-

Phase A – Project Memory & Configuration (before Unit 0)

- ✓ ~~Create GLOSSARY.md~~ tables, columns, foreign keys, URL convention (from doc 4, page 1)
 - ✓ ~~Create .env.example~~ (commit to Git) + actual `.env` (ignored)
 - ✓ ~~Create config/config.php~~ loads `.env` , provides `Config::get()` , `Config::getDbConfig()` , etc.
 - ✓ ~~Decide autoloader:~~
 - ✓ ~~Option A~~ Composer PSR-4 (`composer.json` + `vendor/autoload.php`)
 - ✓ ~~Option B~~ manual `init.php` (not recommended)
 - ✓ ~~Create tests/test_unit.php~~ — template to test each unit in isolation
-

Phase 1 – Core Infrastructure (Units 0–4)

Unit 0 – Database Schema

- ✓ ~~Write sql/database.sql~~ (≥3 related tables: users, products, categories)

- ✓ Include primary keys, auto-increment, foreign keys (`ON DELETE CASCADE/SET NULL`)
- ✓ Add sample data (3-5 rows per table)
- ✓ Execute SQL on MySQL — verify no errors

Unit 1 – Database Singleton (PDO)

- ✓ Create `config/Database.php` (Singleton pattern, PDO, reads from `Config::getDbConfig()`)
- ✓ Set PDO options: exceptions, associative fetch, persistent connection
- ✓ Prevent clone / wakeup
- ✓ Test in isolation: `Database::getInstance()` returns PDO object

Unit 2 – Entity Classes (POPO)

- ✓ Create `models/entities/User.php` (properties, constructor hydration, getters/setters)
- ✓ Create `models/entities/Product.php`
- ✓ Create `models/entities/Category.php`
- ✓ (Later) Create `models/entities/File.php` for Unit 8

Unit 3 – DAO / Manager Classes (CRUD)

- ✓ Create `models/daos/UserDao.php` — `find()` , `findAll()` , `save()` , `delete()`
- ✓ Create `models/daos/ProductDao.php` (same methods)
- ✓ Create `models/daos/CategoryDao.php`
 - ✓ Test one DAO (e.g., `UserDao::findAll()`) via `tests/test_unit.php`

Unit 4 – Router & Front Controller

- ✓ Create `public/index.php` — front controller, reads `$_GET['route']` or `REQUEST_URI`
- ✓ Create `Router.php` (in `/public` or `/config`) with `dispatch()` method
- ✓ Map URLs to `Controller@action` , handle 404
- ✓ Test with a dummy `TestController` (e.g., `index.php?route=test/index`)

Phase 2 – Authentication (Unit 5)

- ✓ Create `controllers/AuthController.php` — actions: `login()` , `logout()` , `dashboard()`
- ✓ Create `utils/Auth.php` — static methods: `isLoggedIn()` , `requireLogin()` , `login($user)` , `logout()`
- ✓ Session handling: `session_start()` , regenerate ID on login
- ✓ Login view (`views/auth/login.php`) — POST to `AuthController@login`
- ✓ Password verification using `password_verify()` (hash stored in DB)

- ✓ Protect a test page with `Auth::requireLogin()`
 - ✓ Test: login → dashboard → logout → redirect to login
-

Phase 3 – CRUD for One Entity (Unit 6)

- ✓ Create `controllers/ProductController.php` — `index`, `show`, `create`, `store`, `edit`, `update`, `delete`
 - ✓ Add `Auth::requireLogin()` to all actions
 - ✓ Create views:
 - `views/product/index.php` (list)
 - `views/product/show.php` (detail)
 - `views/product/create.php` (form)
 - `views/product/edit.php` (edit form)
 - ✓ Implement CSRF token (simple session token) in forms
 - ✓ Test full CRUD cycle (Create, Read, Update, Delete)
-

Phase 4 – File Upload + Admin Management (Units 7 & 8)

Unit 7 – Uploader Utility

- ☐ Create `utils/Uploader.php` – static `upload()` method
- ☐ Validation: file exists, no error, size limit, extension + MIME type
- ☐ Generate unique name (timestamp + random string)
- ☐ Move file to `public/uploads/`, return new filename or throw exception

Unit 8 – Admin File Management

- ☐ Extend SQL schema: add `files` table (`id`, `original_name`, `stored_name`, `mime_type`, `size`, `user_id`, `created_at`)
 - ☐ Create `models/entities/File.php`
 - ☐ Create `models/daos/FileDao.php` — `saveFileRecord()`, `findAll()`, `deleteById()`
 - ☐ Create `controllers/FileManagerController.php` — `index()`, `upload()`, `delete()`
 - ☐ Create views: `views/file_manager/index.php` (list + upload form)
 - ☐ Protect with `Auth::requireLogin()` + optional admin role check
 - ☐ Test: upload a JPG/PDF, see it listed, download, delete
-

Phase 5 – Search Module (Unit 9)

- ☐ Create `controllers/SearchController.php` – action `search()` reads GET params (keywords, category, price min/max)
 - ☐ Create `services/SearchService.php` – dynamic prepared statements, search across at least 2 related tables
 - ☐ Create search form view `views/search/index.php`
 - ☐ Implement pagination (LIMIT / OFFSET)
 - ☐ Display results with links to detail pages
 - ☐ Test: search for existing keyword, empty result, pagination
-

Phase 6 – Documentation & Final Integration (Unit 10)

- ☐ **Generate** `README.md` – title, description, install steps, requirements, config, default credentials, features, folder structure
 - ☐ **Write** `user_guide.pdf` – user manual with screenshots (cahier des charges)
 - ☐ **Write** `ai_report.pdf` – includes the **AI usage table** (see below)
 - ☐ **Full integration test** – create user → login → add product → upload file → search → logout → verify everything works
 - ☐ Asset path fix: ensure uploaded images display correctly (relative vs public URL)
 - ☐ Final commit & push to GitHub (regular commit history visible)
-

AI Reporting Table (to be included in `ai_report.pdf`)

For each major AI interaction, fill a row:

Prompt (what you asked)	AI Response (code / explanation)	Critical Analysis & Adjustments	Justification
<i>Ex: “Create Database Singleton with PDO”</i>	<i>...code...</i>	<i>Added UTF-8 charset, persistent connection</i>	<i>First time implementing Singleton pattern</i>
... add rows as you work ...			

Rule: Log every major generation (SQL, DAO, Router, Auth, CRUD, Uploader, Search) and every debugging session (white screen, PDO exception, session loss, path issues).
No need to log trivial typos.



Generic Debugging Prompts (copy when stuck)

Problem	Prompt to copy to AI
White screen / no error	<i>Enable full error reporting in PHP. Add <code>error_reporting(E_ALL); ini_set('display_errors', 1);</code> at the top of index.php. Then tell me what error appears.</i>
PDO “column not found”	<i>My SQL query uses column names that exist. Here is the query and schema. Fix typos or case mismatches.</i>
Class not found (autoload)	<i>I’m not using an autoloader / using Composer. Here is my index.php with <code>require_once</code> paths. Fix paths so all classes load.</i>
Session lost between pages	<i>I call <code>session_start()</code> on every page. Session disappears after redirect. Show me how to verify session save path and cookie parameters.</i>
POST data empty	<i>My form method is POST but <code>var_dump(\$_POST)</code> is empty. Fix form action or enctype.</i>
Foreign key constraint fails	<i>I try to delete a user with related products. ON DELETE CASCADE should work but fails. Here is my table definition. Fix constraint or delete logic.</i>



Final Submission Checklist (from project brief)

- ☐ GitHub repo with regular commits (both members)
- ☐ README.md – install + SQL dump
- ☐ cahier des charges – presentation, relation diagram, user guide (screenshots)
- ☐ ai_report.pdf – completed table
- ☐ No [cite_start] or [cite: x] left in final docs
- ☐ All 10 units completed and tested individually

Last update: (fill date)

Current progress: x / 68 tasks completed (update manually)